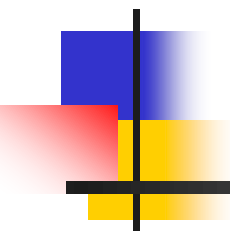
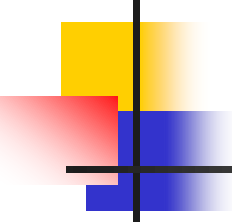


DATA STRUCTURES AND ALGORITHMS



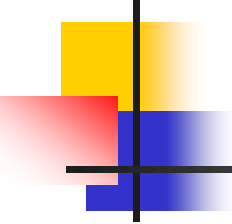
Lecture Notes 9

Pradit Pitaksathienkul



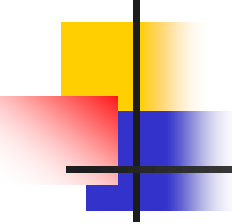
ROAD MAP

- **SORTING**
 - Preliminaries
 - Insertion sort
 - Selection sort
 - Bubble sort



Preliminaries

- The algorithms will all be exchangeable.
- Each will be passed an array containing the elements.
- We will also assume the existence of the "<" and ">" operators.
- Sorting under these conditions is known as *comparison-based* sorting.



Insertion Sort

Original	34	8	64	51	32	21	P. Moved
----------	----	---	----	----	----	----	----------

After $p = 1$	8	34	64	51	32	21	1
---------------	---	----	----	----	----	----	---

After $p = 2$	8	34	64	51	32	21	0
---------------	---	----	----	----	----	----	---

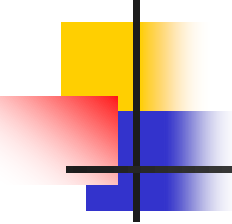
After $p = 3$	8	34	51	64	32	21	1
---------------	---	----	----	----	----	----	---

After $p = 4$	8	32	34	51	64	21	3
---------------	---	----	----	----	----	----	---

After $p = 5$	8	21	32	34	51	64	4
---------------	---	----	----	----	----	----	---

```
def insertion_sort(a):  
    for p in range(1, len(a)):  
        tmp = a[p]  
        j = p  
        while j > 0 and tmp < a[j - 1]:  
            a[j] = a[j - 1]  
            j -= 1  
        a[j] = tmp
```

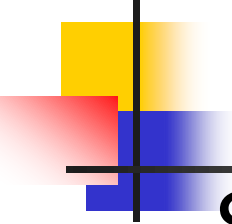
```
//Insertion sort routine
```



3 cases in algorithms

There are 3 cases
when study in algorithms.

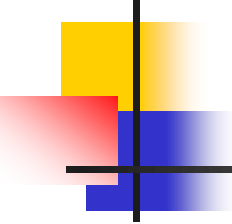
- Best Case
- Worst Case
- Average Case



Selection sort

Selection Sort is a simple comparison-based sorting algorithm that works by dividing the input list into two parts: the sorted part and the unsorted part.

The algorithm repeatedly finds the minimum element from the unsorted part and swaps it with the first element of the unsorted part. This process is repeated until the entire list becomes sorted.



Selection Sort

Original	[64, 25, 12, 22, 11]	min_idx	swap
----------	----------------------	---------	------

For i= 0 ,j= 1	[64, 25, 12, 22, 11]	1	0
----------------	----------------------	---	---

For i= 0 ,j= 2	[64, 25, 12, 22, 11]	2	0
----------------	----------------------	---	---

For i= 0 ,j= 3	[64, 25, 12, 22, 11]	2	0
----------------	----------------------	---	---

For i= 0 ,j= 4	[64, 25, 12, 22, 11]	4	0
----------------	----------------------	---	---

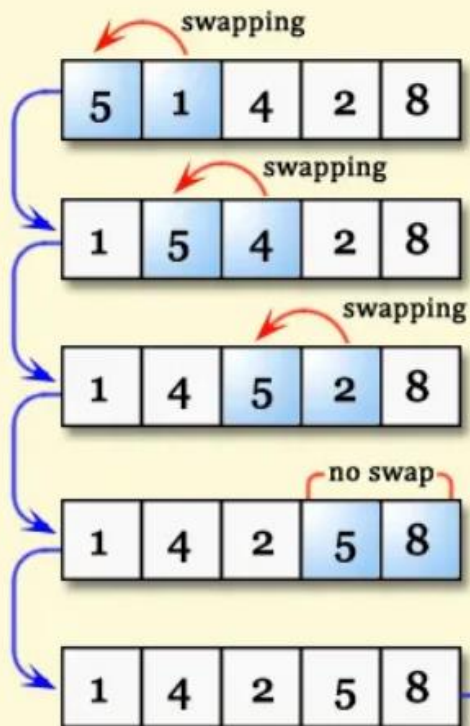
For i= 1 ,j= 2	[11, 25, 12, 22, 64]	2	1
----------------	----------------------	---	---

```
def selection_sort(arr):  
    n = len(arr)  
    for i in range(n):  
        min_idx = i  
        for j in range(i + 1, n):  
            if arr[j] < arr[min_idx]:  
                min_idx = j  
  
        if min_idx != i:  
            temp = arr[i]  
            arr[i] = arr[min_idx]  
            arr[min_idx] = temp
```

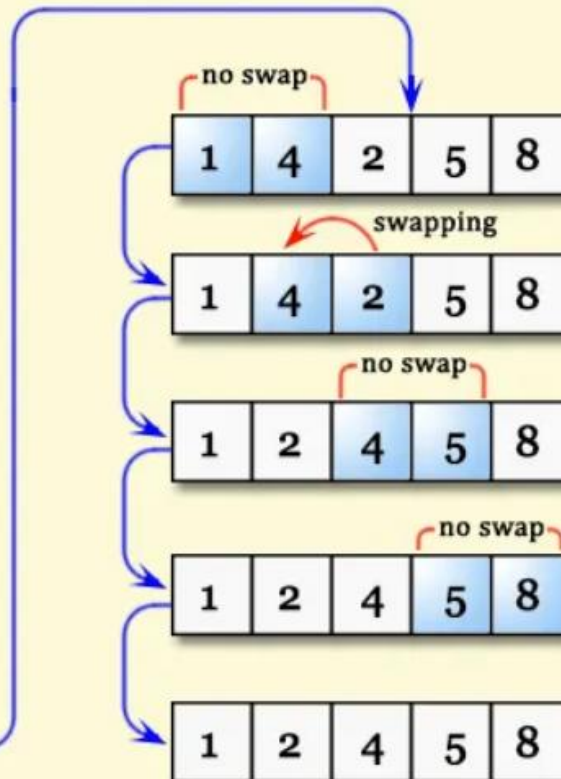
```
//Selection sort routine
```

Bubble Sort

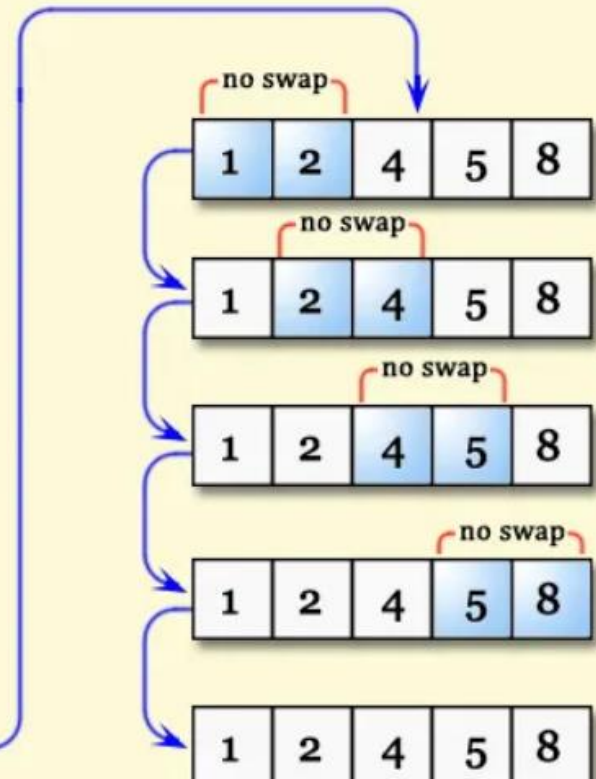
First Pass



Second Pass



Third Pass



```
def bubble_sort(arr):  
    n = len(arr)  
    for i in range(n - 1):  
        last_index = n - 1 - i  
        for j in range(last_index):  
            if arr[j] > arr[j + 1]:  
                temp = arr[j]  
                arr[j] = arr[j + 1]  
                arr[j + 1] = temp
```

```
//Bubble sort routine
```

Bubble Sort requires at most $n-1$ passes
for n elements